

Corewar

Generated by Doxygen 1.9.8

1 File Index	1
1.1 File List	1
2 File Documentation	3
2.1 main.c File Reference	3
2.1.1 Function Documentation	4
2.1.1.1 execute_cycle()	4
2.1.1.2 launch_vm()	4
2.1.1.3 main()	4
2.1.1.4 make_instruction()	5
2.1.1.5 print_help()	5
2.1.1.6 step_forward()	5
2.2 src/champions/create_champions.c File Reference	6
2.2.1 Function Documentation	6
2.2.1.1 create_champions()	6
2.2.1.2 get_champion_infos()	7
2.2.1.3 load_champion_body_to_buffer()	7
2.2.1.4 reverse_bytes()	7
2.3 src/free_functions/free.c File Reference	8
2.3.1 Function Documentation	8
2.3.1.1 free_struct()	8
2.4 src/instructions_code/add.c File Reference	9
2.4.1 Function Documentation	9
2.4.1.1 affect_value_add()	9
2.4.1.2 make_add()	10
2.5 src/instructions_code/aff.c File Reference	10
2.5.1 Function Documentation	11
2.5.1.1 make_aff()	11
2.5.1.2 recover_registre()	11
2.6 src/instructions_code/and.c File Reference	12
2.6.1 Function Documentation	12
2.6.1.1 affect_value_and()	12
2.6.1.2 make_and()	13
2.7 src/instructions_code/check_types.c File Reference	13
2.7.1 Function Documentation	14
2.7.1.1 check_if_indirect()	14
2.7.1.2 check_type()	14
2.7.1.3 get_arg_type()	15
2.7.1.4 recover_indirect()	15
2.8 src/instructions_code/fork.c File Reference	16
2.8.1 Function Documentation	16
2.8.1.1 add_new_process()	16

2.8.1.2 make_fork()	17
2.9 src/instructions_code/ld.c File Reference	17
2.9.1 Function Documentation	18
2.9.1.1 affect_value_ld()	18
2.9.1.2 make_ld()	18
2.9.1.3 print_register_change()	19
2.10 src/instructions_code/ldi.c File Reference	19
2.10.1 Function Documentation	20
2.10.1.1 affect_value_ldi()	20
2.10.1.2 make_ldi()	20
2.10.1.3 read_value()	20
2.11 src/instructions_code/lfork.c File Reference	21
2.11.1 Function Documentation	22
2.11.1.1 make_lfork()	22
2.12 src/instructions_code/live.c File Reference	22
2.12.1 Function Documentation	23
2.12.1.1 make_live()	23
2.13 src/instructions_code/lld.c File Reference	23
2.13.1 Function Documentation	24
2.13.1.1 affect_value_lld()	24
2.13.1.2 make_lld()	24
2.14 src/instructions_code/lldi.c File Reference	24
2.14.1 Function Documentation	25
2.14.1.1 affect_value_lldi()	25
2.14.1.2 make_lldi()	26
2.15 src/instructions_code/modif_proces.c File Reference	26
2.15.1 Function Documentation	27
2.15.1.1 calculate_new_pc()	27
2.15.1.2 change_carry()	27
2.15.1.3 recover_dir_size()	27
2.15.1.4 recover_direct_value()	28
2.16 src/instructions_code/or.c File Reference	28
2.16.1 Function Documentation	29
2.16.1.1 affect_value_or()	29
2.16.1.2 make_or()	29
2.17 src/instructions_code/st.c File Reference	30
2.17.1 Function Documentation	30
2.17.1.1 affect_value_st()	30
2.17.1.2 make_st()	31
2.17.1.3 stock_value()	31
2.18 src/instructions_code/sti.c File Reference	31
2.18.1 Function Documentation	32

2.18.1.1 make_sti()	32
2.18.1.2 recup_value_sti()	33
2.18.1.3 stock_value_i()	33
2.19 src/instructions_code/subb.c File Reference	33
2.19.1 Function Documentation	34
2.19.1.1 affect_value_subb()	34
2.19.1.2 make_sub()	35
2.20 src/instructions_code/xor.c File Reference	35
2.20.1 Function Documentation	36
2.20.1.1 affect_value_xor()	36
2.20.1.2 make_xor()	36
2.21 src/instructions_code/zjmp.c File Reference	36
2.21.1 Function Documentation	37
2.21.1.1 make_zjmp()	37
2.21.1.2 take_one_arg()	37
2.22 src/parsing/flags.c File Reference	38
2.22.1 Function Documentation	39
2.22.1.1 assign_missing_addresses()	39
2.22.1.2 assign_missing_numbers()	39
2.22.1.3 change_load_adress()	39
2.22.1.4 change_number_cycle()	39
2.22.1.5 change_prog_number()	40
2.22.1.6 parsing_flags()	40
2.22.1.7 recover_cycle()	40
2.22.1.8 recup_load_and_number()	41
2.22.1.9 verif_cor()	41
2.22.1.10 verif_end_parsing()	42
2.23 src/processes/create_processes.c File Reference	42
2.23.1 Function Documentation	42
2.23.1.1 affect_value_process()	42
2.23.1.2 initialize_process()	43
2.24 src/virtual_machine/check_winner.c File Reference	43
2.24.1 Function Documentation	44
2.24.1.1 check_delta()	44
2.24.1.2 check_hunger_games()	44
2.24.1.3 print_winner()	44
2.25 src/virtual_machine/init_arena.c File Reference	45
2.25.1 Function Documentation	45
2.25.1.1 create_arena_with_player()	45
2.25.1.2 verif_size_arena()	46

Chapter 1

File Index

1.1 File List

Here is a list of all files with brief descriptions:

main.c	3
src/champions/ create_champions.c	6
src/free_functions/ free.c	8
src/instructions_code/ add.c	9
src/instructions_code/ aff.c	10
src/instructions_code/ and.c	12
src/instructions_code/ check_types.c	13
src/instructions_code/ fork.c	16
src/instructions_code/ ld.c	17
src/instructions_code/ ldi.c	19
src/instructions_code/ lfork.c	21
src/instructions_code/ live.c	22
src/instructions_code/ lld.c	23
src/instructions_code/ lldi.c	24
src/instructions_code/ modif_proces.c	26
src/instructions_code/ or.c	28
src/instructions_code/ st.c	30
src/instructions_code/ sti.c	31
src/instructions_code/ subb.c	33
src/instructions_code/ xor.c	35
src/instructions_code/ zjmp.c	36
src/parsing/ flags.c	38
src/processes/ create_processes.c	42
src/virtual_machine/ check_winner.c	43
src/virtual_machine/ init_arena.c	45

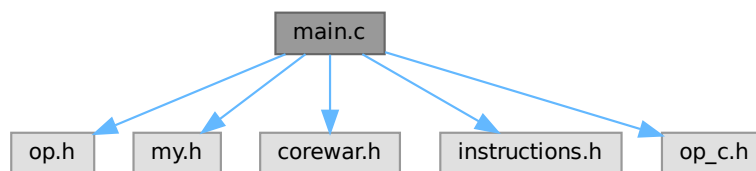
Chapter 2

File Documentation

2.1 main.c File Reference

```
#include "op.h"  
#include "my.h"  
#include "corewar.h"  
#include "instructions.h"  
#include "op_c.h"
```

Include dependency graph for main.c:



Functions

- int [print_help](#) (void)
Prints the help message for the Corewar program.
- int [make_instruction](#) (processes_t *process_actual, corewar_t *global)
Executes the instruction for the current process.
- static void [step_forward](#) (processes_t *proc, corewar_t *global)
Advances the process to the next instruction.
- static void [execute_cycle](#) (corewar_t *global)
Executes a single cycle of the Corewar virtual machine.
- static void [launch_vm](#) (corewar_t *global)
Launches the Corewar virtual machine.
- int [main](#) (int ac, char **av)
Entry point for the Corewar program.

2.1.1 Function Documentation

2.1.1.1 `execute_cycle()`

```
static void execute_cycle (
    corewar_t * global ) [static]
```

Executes a single cycle of the Corewar virtual machine.

This function iterates through all processes and executes their instructions. If a process is waiting, its wait time is decremented.

Parameters

<i>global</i>	Pointer to the corewar global structure.
---------------	--

2.1.1.2 `launch_vm()`

```
static void launch_vm (
    corewar_t * global ) [static]
```

Launches the Corewar virtual machine.

This function runs the Corewar game loop, executing cycles until all processes are terminated or the specified number of cycles is reached. It also manages the cycle-to-die checks and prints the winner at the end.

Parameters

<i>global</i>	Pointer to the corewar global structure.
---------------	--

2.1.1.3 `main()`

```
int main (
    int ac,
    char ** av )
```

Entry point for the Corewar program.

This function initializes the Corewar game, parses the arguments, creates the players and processes, and starts the virtual machine.

Parameters

<i>ac</i>	Number of arguments.
<i>av</i>	Array of arguments.

Returns

0 on success, 84 if an error occurs.

2.1.1.4 make_instruction()

```
int make_instruction (
    processes_t * process_actual,
    corewar_t * global )
```

Executes the instruction for the current process.

This function checks the instruction at the program counter of the current process and executes it if it matches a valid instruction. If no valid instruction is found, the program counter is incremented.

Parameters

<i>process_actual</i>	Pointer to the current process structure.
<i>global</i>	Pointer to the corewar global structure.

Returns

0 on success, 1 if the instruction is invalid.

2.1.1.5 print_help()

```
int print_help (
    void )
```

Prints the help message for the Corewar program.

This function displays the usage and description of the Corewar program, including the available flags and their purposes.

Returns

0 on success.

2.1.1.6 step_forward()

```
static void step_forward (
    processes_t * proc,
    corewar_t * global ) [static]
```

Advances the process to the next instruction.

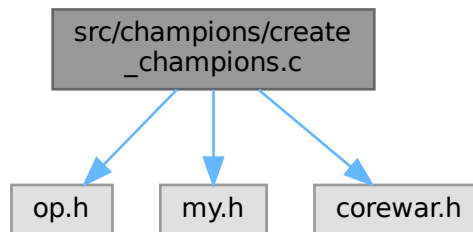
This function executes the current instruction for the process. If the instruction is invalid, the program counter is incremented.

Parameters

<i>proc</i>	Pointer to the current process structure.
<i>global</i>	Pointer to the corewar global structure.

2.2 src/champions/create_champions.c File Reference

```
#include "op.h"
#include "my.h"
#include "corewar.h"
Include dependency graph for create_champions.c:
```



Functions

- void [reverse_bytes](#) (void *ptr, size_t size)
Reverses the bytes of a given memory block.
- static int [load_champion_body_to_buffer](#) (FILE *fd, player_t *player, int i)
Loads the champion's body into a buffer.
- int [get_champion_infos](#) (player_t player[4], int i)
Retrieves information about a champion from its file.
- int [create_champions](#) (player_t player[4], corewar_t *globals)
Creates champions by loading their information and code.

2.2.1 Function Documentation

2.2.1.1 create_champions()

```
int create_champions (
    player_t player[4],
    corewar_t * globals )
```

Creates champions by loading their information and code.

This function initializes the champions by reading their metadata and binary code from their respective files. It iterates over all the champions specified in the `globals` structure and stores their information in the `player` array.

Parameters

<i>player</i>	Array of player structures.
<i>globals</i>	Pointer to the corewar global structure containing the number of champions and other global data.

Returns

0 on success, 84 on failure (e.g., invalid champion file or initialization error).

2.2.1.2 get_champion_infos()

```
int get_champion_infos (
    player_t player[4],
    int i )
```

Retrieves information about a champion from its file.

This function reads the header of a champion file to extract metadata such as the program name, comment, and size. It also verifies the magic number to ensure the file is valid. The champion's binary code is then loaded into memory.

Parameters

<i>player</i>	Array of player structures.
<i>i</i>	Index of the current player.

Returns

0 on success, 84 on failure (e.g., invalid file, memory allocation failure, or read error).

2.2.1.3 load_champion_body_to_buffer()

```
static int load_champion_body_to_buffer (
    FILE * fd,
    player_t * player,
    int i ) [static]
```

Loads the champion's body into a buffer.

This function reads the binary code of a champion from a file and stores it in the player's `code` buffer. The size of the buffer is determined by the `prog_size` field of the player structure.

Parameters

<i>fd</i>	File descriptor of the champion file.
<i>player</i>	Array of player structures.
<i>i</i>	Index of the current player.

Returns

0 on success, 84 on failure (e.g., memory allocation failure or read error).

2.2.1.4 reverse_bytes()

```
void reverse_bytes (
```

```
void * ptr,
size_t size )
```

Reverses the bytes of a given memory block.

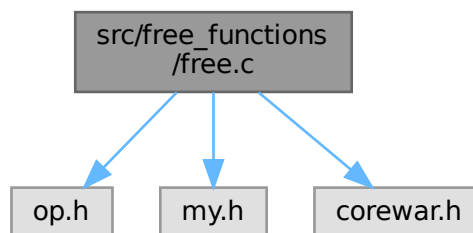
This function takes a pointer to a memory block and reverses the order of its bytes. It is typically used to handle endianness issues when reading or writing binary data.

Parameters

<i>ptr</i>	Pointer to the memory block to reverse.
<i>size</i>	Size of the memory block in bytes.

2.3 src/free_functions/free.c File Reference

```
#include "op.h"
#include "my.h"
#include "corewar.h"
Include dependency graph for free.c:
```



Functions

- void [free_struct](#) (corewar_t global, processes_t *start_process, player_t player[4])
Frees all allocated memory for the corewar structures.

2.3.1 Function Documentation

2.3.1.1 free_struct()

```
void free_struct (
    corewar_t global,
    processes_t * start_process,
    player_t player[4] )
```

Frees all allocated memory for the corewar structures.

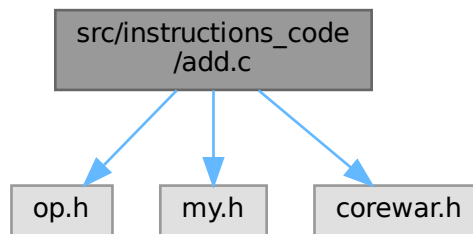
This function releases all dynamically allocated memory associated with the corewar game, including the arena, player data, and the linked list of processes. It ensures that no memory leaks occur when the program terminates.

Parameters

<i>global</i>	The corewar global structure containing the arena and other global data.
<i>start_process</i>	Pointer to the head of the linked list of processes.
<i>player</i>	Array of player structures containing champion data.

2.4 src/instructions_code/add.c File Reference

```
#include "op.h"
#include "my.h"
#include "corewar.h"
Include dependency graph for add.c:
```



Functions

- int [affect_value_add](#) (processes_t *process, params_values_t *values, corewar_t *global)
Executes the add operation for a process.
- int [make_add](#) (corewar_t *global, processes_t *process)
Handles the add instruction for a process.

2.4.1 Function Documentation

2.4.1.1 affect_value_add()

```
int affect_value_add (
    processes_t * process,
    params_values_t * values,
    corewar_t * global )
```

Executes the add operation for a process.

This function performs the addition of two register values and stores the result in a third register. It also updates the program counter and the carry flag of the process. The function ensures that the operation is valid by checking the argument types and waiting for the required number of cycles before execution.

Parameters

<i>process</i>	Pointer to the current process structure.
<i>values</i>	Pointer to the structure containing the parameter values.
<i>global</i>	Pointer to the corewar global structure.

Returns

0 on success, 1 if the operation is invalid.

2.4.1.2 make_add()

```
int make_add (
    corewar_t * global,
    processes_t * process )
```

Handles the add instruction for a process.

This function decodes the add instruction from the arena, validates the argument types, and calls the function to execute the operation. The add instruction adds the values of two registers and stores the result in a third register.

Parameters

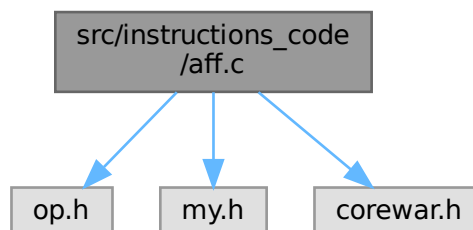
<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.

Returns

0 on success, 1 if the operation is invalid.

2.5 src/instructions_code/aff.c File Reference

```
#include "op.h"
#include "my.h"
#include "corewar.h"
Include dependency graph for aff.c:
```



Functions

- int [recover_registre](#) (int position, corewar_t *global, processes_t *process)
Retrieves the value stored in a register.
- int [make_aff](#) (corewar_t *global, processes_t *process)
Executes the aff instruction for a process.

2.5.1 Function Documentation

2.5.1.1 make_aff()

```
int make_aff (
    corewar_t * global,
    processes_t * process )
```

Executes the aff instruction for a process.

This function decodes the aff instruction, retrieves the value from the specified register, and prints it as an ASCII character. It also manages the process's readiness and updates the program counter.

Parameters

<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.

Returns

0 on success, 1 if the operation is invalid.

2.5.1.2 recover_registre()

```
int recover_registre (
    int position,
    corewar_t * global,
    processes_t * process )
```

Retrieves the value stored in a register.

This function retrieves the value of a register based on its position in the arena. If the register index is invalid, it returns -1.

Parameters

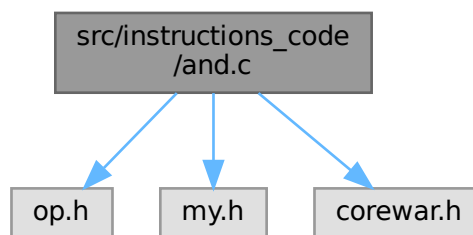
<i>position</i>	The position in the arena to read the register index.
<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.

Returns

The value of the register, or -1 if the index is invalid.

2.6 src/instructions_code/and.c File Reference

```
#include "op.h"
#include "my.h"
#include "corewar.h"
Include dependency graph for and.c:
```

**Functions**

- int [affect_value_and](#) (processes_t *process, params_values_t *values, corewar_t *global)
Executes the AND operation for a process.
- int [make_and](#) (corewar_t *global, processes_t *process)
Handles the AND instruction for a process.

2.6.1 Function Documentation**2.6.1.1 affect_value_and()**

```
int affect_value_and (
    processes_t * process,
    params_values_t * values,
    corewar_t * global )
```

Executes the AND operation for a process.

This function performs a bitwise AND operation between two values and stores the result in a specified register. It also updates the program counter and the carry flag of the process. The function ensures that the operation is valid by checking the argument types and waiting for the required number of cycles before execution.

Parameters

<i>process</i>	Pointer to the current process structure.
<i>values</i>	Pointer to the structure containing the parameter values.
<i>global</i>	Pointer to the corewar global structure.

Returns

0 on success, 1 if the operation is invalid.

2.6.1.2 make_and()

```
int make_and (
    corewar_t * global,
    processes_t * process )
```

Handles the AND instruction for a process.

This function decodes the AND instruction from the arena, validates the argument types, and calls the function to execute the operation. The AND instruction performs a bitwise AND between two values and stores the result in a register.

Parameters

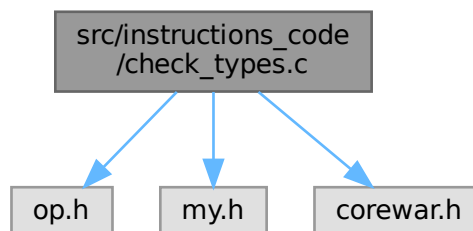
<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.

Returns

0 on success, 1 if the operation is invalid.

2.7 src/instructions_code/check_types.c File Reference

```
#include "op.h"
#include "my.h"
#include "corewar.h"
Include dependency graph for check_types.c:
```

**Functions**

- int [get_arg_type](#) (unsigned char coding_byte, int pos)

Retrieves the type of an argument from the coding byte.

- void `check_if_indirect` (params_values_t *values, corewar_t *global, processes_t *process, int i)

Handles indirect arguments and retrieves their values.

- int `recover_indirect` (corewar_t *global, int pc, int decal, params_values_t values)

Retrieves the value of an indirect argument.

- int `check_type` (params_values_t *values, corewar_t *global, processes_t *process, int i)

Checks the type of an argument and retrieves its value.

2.7.1 Function Documentation

2.7.1.1 `check_if_indirect()`

```
void check_if_indirect (
    params_values_t * values,
    corewar_t * global,
    processes_t * process,
    int i )
```

Handles indirect arguments and retrieves their values.

This function checks if an argument is of type indirect. If so, it calculates the offset and retrieves the corresponding value from the arena. The position is updated accordingly.

Parameters

<i>values</i>	Pointer to the structure containing the parameter values.
<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.
<i>i</i>	The index of the argument to check.

2.7.1.2 `check_type()`

```
int check_type (
    params_values_t * values,
    corewar_t * global,
    processes_t * process,
    int i )
```

Checks the type of an argument and retrieves its value.

This function validates the type of an argument (register, direct, or indirect) and retrieves its value. It updates the position in the arena and ensures that the argument is valid.

Parameters

<i>values</i>	Pointer to the structure containing the parameter values.
<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.
<i>i</i>	The index of the argument to check.

Returns

0 on success, 1 if the argument is invalid.

2.7.1.3 get_arg_type()

```
int get_arg_type (
    unsigned char coding_byte,
    int pos )
```

Retrieves the type of an argument from the coding byte.

This function extracts the type of an argument (register, direct, or indirect) from the coding byte based on its position. It ensures that the position is valid and returns the corresponding type.

Parameters

<i>coding_byte</i>	The coding byte containing argument type information.
<i>pos</i>	The position of the argument (0, 1, or 2).

Returns

The type of the argument (T_REG, T_DIR, T_IND), or 0 if invalid.

2.7.1.4 recover_indirect()

```
int recover_indirect (
    corewar_t * global,
    int pc,
    int decal,
    params_values_t values )
```

Retrieves the value of an indirect argument.

This function calculates the address of an indirect argument based on the program counter and the offset. It retrieves the value stored at the calculated address in the arena.

Parameters

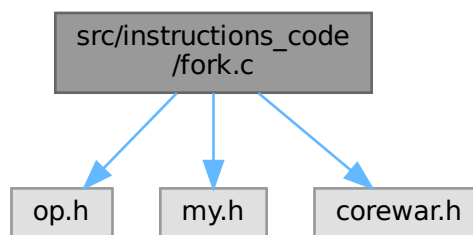
<i>global</i>	Pointer to the corewar global structure.
<i>pc</i>	The program counter of the process.
<i>decal</i>	The offset to calculate the address.
<i>values</i>	The structure containing the parameter values.

Returns

The value of the indirect argument.

2.8 src/instructions_code/fork.c File Reference

```
#include "op.h"
#include "my.h"
#include "corewar.h"
Include dependency graph for fork.c:
```

**Functions**

- `processes_t * add\_new\_process (corewar_t *global, processes_t *process, int new_pc)`
Creates a new process and initializes its state.
- `int make\_fork (corewar_t *global, processes_t *process)`
Executes the fork instruction for a process.

2.8.1 Function Documentation**2.8.1.1 add_new_process()**

```
processes_t * add_new_process (
    corewar_t * global,
    processes_t * process,
    int new_pc )
```

Creates a new process and initializes its state.

This function allocates memory for a new process, copies the state of the parent process, and sets the program counter to the specified value. The new process is initialized with the same register values and other attributes as the parent process.

Parameters

<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the parent process structure.
<i>new_pc</i>	The program counter for the new process.

Returns

Pointer to the newly created process, or NULL on failure.

2.8.1.2 make_fork()

```
int make_fork (
    corewar_t * global,
    processes_t * process )
```

Executes the fork instruction for a process.

This function decodes the fork instruction, calculates the new program counter for the child process, and creates a new process. The child process is added to the end of the process list. The function also manages the readiness of the parent process and updates its program counter.

Parameters

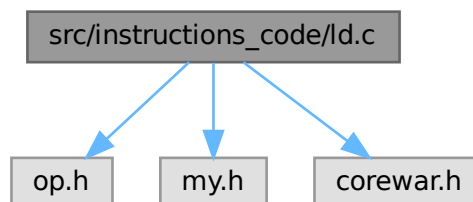
<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.

Returns

0 on success, or 1 if the operation is invalid.

2.9 src/instructions_code/ld.c File Reference

```
#include "op.h"
#include "my.h"
#include "corewar.h"
Include dependency graph for ld.c:
```

**Functions**

- int [affect_value_ld](#) (params_values_t *values, processes_t *process, corewar_t *global)
Validates and retrieves the values for the ld instruction.

- void `print_register_change` (`processes_t *proc`, `params_values_t *values`)
Updates the carry flag and program counter for the ld instruction.
- int `make_ld` (`corewar_t *global`, `processes_t *process`)
Executes the ld instruction for a process.

2.9.1 Function Documentation

2.9.1.1 `affect_value_ld()`

```
int affect_value_ld (
    params_values_t * values,
    processes_t * process,
    corewar_t * global )
```

Validates and retrieves the values for the ld instruction.

This function checks the argument types for the ld instruction and retrieves their values. It ensures that the arguments are valid and updates the position in the arena accordingly.

Parameters

<i>values</i>	Pointer to the structure containing the parameter values.
<i>process</i>	Pointer to the current process structure.
<i>global</i>	Pointer to the corewar global structure.

Returns

0 on success, 1 if the arguments are invalid.

2.9.1.2 `make_ld()`

```
int make_ld (
    corewar_t * global,
    processes_t * process )
```

Executes the ld instruction for a process.

This function decodes the ld instruction, validates the argument types, retrieves their values, and loads the value into the specified register. It also manages the readiness of the process and updates the program counter and carry flag.

Parameters

<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.

Returns

0 on success, 1 if the operation is invalid.

2.9.1.3 print_register_change()

```
void print_register_change (
    processes_t * proc,
    params_values_t * values )
```

Updates the carry flag and program counter for the ldi instruction.

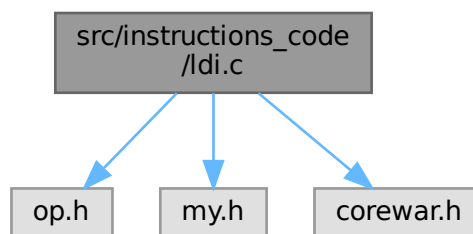
This function updates the carry flag based on the value loaded into the register and increments the program counter to the next instruction.

Parameters

<i>proc</i>	Pointer to the current process structure.
<i>values</i>	Pointer to the structure containing the parameter values.

2.10 src/instructions_code/ldi.c File Reference

```
#include "op.h"
#include "my.h"
#include "corewar.h"
Include dependency graph for ldi.c:
```



Functions

- int [read_value](#) (params_values_t values, processes_t *process, corewar_t *global, bool is_ldi)
Reads a value for the ldi or lldi instruction.
- int [affect_value_ldi](#) (params_values_t *values, corewar_t *global, processes_t *process)
Validates and retrieves the values for the ldi instruction.
- int [make_ldi](#) (corewar_t *global, processes_t *process)
Executes the ldi instruction for a process.

2.10.1 Function Documentation

2.10.1.1 `affect_value_ldi()`

```
int affect_value_ldi (
    params_values_t * values,
    corewar_t * global,
    processes_t * process )
```

Validates and retrieves the values for the ldi instruction.

This function checks the argument types for the ldi instruction and retrieves their values. It ensures that the arguments are valid and updates the position in the arena accordingly.

Parameters

<i>values</i>	Pointer to the structure containing the parameter values.
<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.

Returns

0 on success, 1 if the arguments are invalid.

2.10.1.2 `make_ldi()`

```
int make_ldi (
    corewar_t * global,
    processes_t * process )
```

Executes the ldi instruction for a process.

This function decodes the ldi instruction, validates the argument types, retrieves their values, and loads the value into the specified register. It also manages the readiness of the process and updates the program counter and carry flag.

Parameters

<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.

Returns

0 on success, 1 if the operation is invalid.

2.10.1.3 `read_value()`

```
int read_value (
    params_values_t values,
```

```

processes_t * process,
corewar_t * global,
bool is_ldi )

```

Reads a value for the ldi or lldi instruction.

This function calculates the offset based on the sum of the first two arguments and retrieves the value from the arena. If the instruction is lldi, the offset is not restricted by IDX_MOD.

Parameters

<i>values</i>	The structure containing the parameter values.
<i>process</i>	Pointer to the current process structure.
<i>global</i>	Pointer to the corewar global structure.
<i>is_ldi</i>	Boolean indicating if the instruction is lldi.

Returns

The value retrieved from the arena.

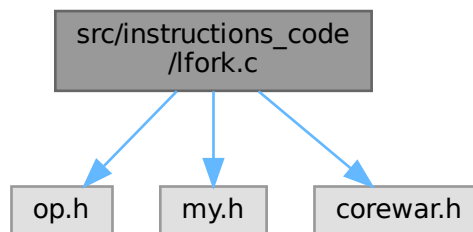
2.11 src/instructions_code/lfork.c File Reference

```

#include "op.h"
#include "my.h"
#include "corewar.h"

```

Include dependency graph for lfork.c:



Functions

- int [make_lfork](#) (corewar_t *global, processes_t *process)
Executes the lfork instruction for a process.

2.11.1 Function Documentation

2.11.1.1 `make_lfork()`

```
int make_lfork (
    corewar_t * global,
    processes_t * process )
```

Executes the lfork instruction for a process.

This function decodes the lfork instruction, calculates the new program counter for the child process, and creates a new process. Unlike the fork instruction, the offset for lfork is not restricted by `IDX_MOD`. The child process is added to the end of the process list. The function also manages the readiness of the parent process and updates its program counter.

Parameters

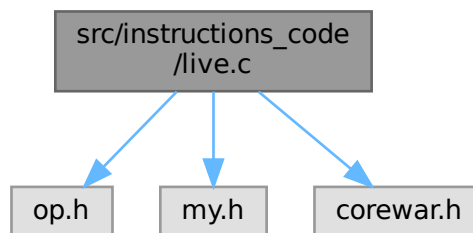
<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.

Returns

0 on success, or 1 if the operation is invalid.

2.12 `src/instructions_code/live.c` File Reference

```
#include "op.h"
#include "my.h"
#include "corewar.h"
Include dependency graph for live.c:
```



Functions

- int [make_live](#) (corewar_t *global, processes_t *process)
Executes the live instruction for a process.

2.12.1 Function Documentation

2.12.1.1 make_live()

```
int make_live (
    corewar_t * global,
    processes_t * process )
```

Executes the live instruction for a process.

This function decodes the live instruction, retrieves the player number from the arena, and marks the corresponding player as alive. It updates the live cycle of the process and the player, increments the global live counter, and sets the last alive player. If the player number matches a champion, a message is printed to indicate that the player is alive.

Parameters

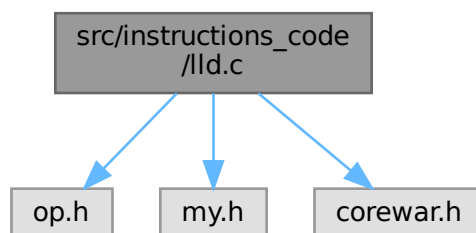
<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.

Returns

0 on success.

2.13 src/instructions_code/lld.c File Reference

```
#include "op.h"
#include "my.h"
#include "corewar.h"
Include dependency graph for lld.c:
```



Functions

- int [affect_value_lld](#) (params_values_t *values, processes_t *process, corewar_t *global)
Validates and retrieves the values for the lld instruction.
- int [make_lld](#) (corewar_t *global, processes_t *process)
Executes the lld instruction for a process.

2.13.1 Function Documentation

2.13.1.1 `affect_value_lld()`

```
int affect_value_lld (
    params_values_t * values,
    processes_t * process,
    corewar_t * global )
```

Validates and retrieves the values for the lld instruction.

This function checks the argument types for the lld instruction and retrieves their values. It ensures that the arguments are valid and updates the position in the arena accordingly.

Parameters

<i>values</i>	Pointer to the structure containing the parameter values.
<i>process</i>	Pointer to the current process structure.
<i>global</i>	Pointer to the corewar global structure.

Returns

0 on success, 1 if the arguments are invalid.

2.13.1.2 `make_lld()`

```
int make_lld (
    corewar_t * global,
    processes_t * process )
```

Executes the lld instruction for a process.

This function decodes the lld instruction, validates the argument types, retrieves their values, and loads the value into the specified register. Unlike the ld instruction, the lld instruction does not restrict the offset by `IDX_MOD`. It also manages the readiness of the process and updates the program counter and carry flag.

Parameters

<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.

Returns

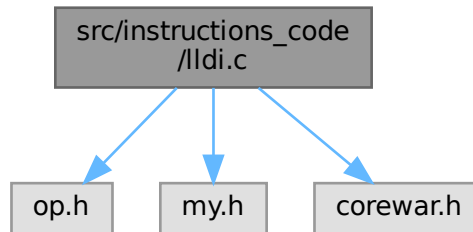
0 on success, 1 if the operation is invalid.

2.14 `src/instructions_code/lldi.c` File Reference

```
#include "op.h"
#include "my.h"
```

```
#include "corewar.h"
```

Include dependency graph for lldi.c:



Functions

- int [affect_value_lldi](#) (params_values_t *values, corewar_t *global, processes_t *process)
Validates and retrieves the values for the lldi instruction.
- int [make_lldi](#) (corewar_t *global, processes_t *process)
Executes the lldi instruction for a process.

2.14.1 Function Documentation

2.14.1.1 [affect_value_lldi\(\)](#)

```
int affect_value_lldi (
    params_values_t * values,
    corewar_t * global,
    processes_t * process )
```

Validates and retrieves the values for the lldi instruction.

This function checks the argument types for the lldi instruction and retrieves their values. It ensures that the arguments are valid and updates the position in the arena accordingly.

Parameters

<i>values</i>	Pointer to the structure containing the parameter values.
<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.

Returns

0 on success, 1 if the arguments are invalid.

2.14.1.2 make_lldi()

```
int make_lldi (
    corewar_t * global,
    processes_t * process )
```

Executes the lldi instruction for a process.

This function decodes the lldi instruction, validates the argument types, retrieves their values, and loads the value into the specified register. Unlike the ldi instruction, the lldi instruction does not restrict the offset by IDX_MOD. It also manages the readiness of the process and updates the program counter and carry flag.

Parameters

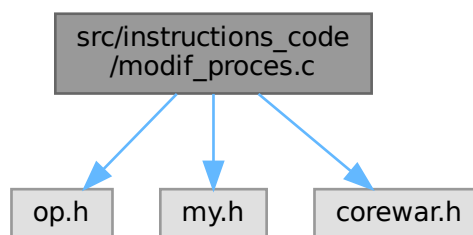
<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.

Returns

0 on success, 1 if the operation is invalid.

2.15 src/instructions_code/modif_proces.c File Reference

```
#include "op.h"
#include "my.h"
#include "corewar.h"
Include dependency graph for modif_proces.c:
```



Functions

- int [recover_dir_size](#) (int command)
Retrieves the size of a direct argument for a command.
- int [calculate_new_pc](#) (params_values_t values, int number_args, int command)
Calculates the new program counter for a process.
- void [change_carry](#) (int data, processes_t *proces)
Updates the carry flag for a process.
- int [recover_direct_value](#) (corewar_t *global, int position, int size)
Retrieves a direct value from the arena.

2.15.1 Function Documentation

2.15.1.1 calculate_new_pc()

```
int calculate_new_pc (
    params_values_t values,
    int number_args,
    int command )
```

Calculates the new program counter for a process.

This function calculates the number of bytes to move the program counter forward based on the argument types and the command. It ensures that the program counter is updated correctly after an instruction is executed.

Parameters

<i>values</i>	The structure containing the parameter values.
<i>number_args</i>	The number of arguments for the instruction.
<i>command</i>	The command code being executed.

Returns

The number of bytes to move the program counter forward.

2.15.1.2 change_carry()

```
void change_carry (
    int data,
    processes_t * proces )
```

Updates the carry flag for a process.

This function sets the carry flag of a process based on the value of the data. If the data is 0, the carry flag is set to 1. Otherwise, it is set to 0.

Parameters

<i>data</i>	The value to check.
<i>proces</i>	Pointer to the current process structure.

2.15.1.3 recover_dir_size()

```
int recover_dir_size (
    int command )
```

Retrieves the size of a direct argument for a command.

This function determines the size of a direct argument (T_DIR) based on the command. Some commands use a 2-byte direct argument, while others use a 4-byte direct argument.

Parameters

<i>command</i>	The command code to check.
----------------	----------------------------

Returns

The size of the direct argument (2 or 4 bytes).

2.15.1.4 recover_direct_value()

```
int recover_direct_value (
    corewar_t * global,
    int position,
    int size )
```

Retrieves a direct value from the arena.

This function reads a direct value (T_DIR) from the arena at the specified position. The size of the value can be either 2 bytes or 4 bytes, depending on the instruction.

Parameters

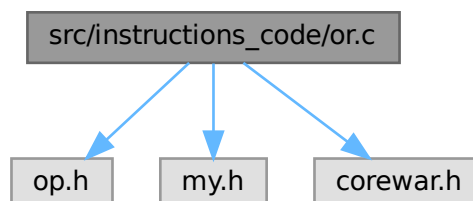
<i>global</i>	Pointer to the corewar global structure.
<i>position</i>	The position in the arena to read from.
<i>size</i>	The size of the value to retrieve (2 or 4 bytes).

Returns

The retrieved direct value.

2.16 src/instructions_code/or.c File Reference

```
#include "op.h"
#include "my.h"
#include "corewar.h"
Include dependency graph for or.c:
```



Functions

- int `affect_value_or` (`processes_t *process`, `params_values_t *values`, `corewar_t *global`)
Executes the OR operation for a process.
- int `make_or` (`corewar_t *global`, `processes_t *process`)
Handles the OR instruction for a process.

2.16.1 Function Documentation

2.16.1.1 `affect_value_or()`

```
int affect_value_or (
    processes_t * process,
    params_values_t * values,
    corewar_t * global )
```

Executes the OR operation for a process.

This function performs a bitwise OR operation between two values and stores the result in a specified register. It also updates the program counter and the carry flag of the process. The function ensures that the operation is valid by checking the argument types and waiting for the required number of cycles before execution.

Parameters

<i>process</i>	Pointer to the current process structure.
<i>values</i>	Pointer to the structure containing the parameter values.
<i>global</i>	Pointer to the corewar global structure.

Returns

0 on success, 1 if the operation is invalid.

2.16.1.2 `make_or()`

```
int make_or (
    corewar_t * global,
    processes_t * process )
```

Handles the OR instruction for a process.

This function decodes the OR instruction from the arena, validates the argument types, and calls the function to execute the operation. The OR instruction performs a bitwise OR between two values and stores the result in a register.

Parameters

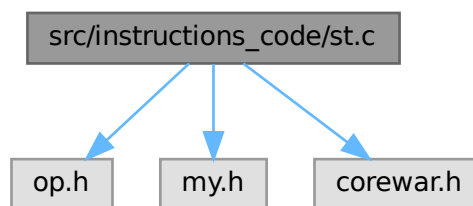
<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.

Returns

0 on success, 1 if the operation is invalid.

2.17 src/instructions_code/st.c File Reference

```
#include "op.h"
#include "my.h"
#include "corewar.h"
Include dependency graph for st.c:
```



Functions

- int [affect_value_st](#) (params_values_t *values, corewar_t *global, processes_t *process)
Validates and retrieves the values for the st instruction.
- void [stock_value](#) (corewar_t *global, processes_t *process, params_values_t values)
Stores a value in the arena or a register.
- int [make_st](#) (corewar_t *global, processes_t *process)
Executes the st instruction for a process.

2.17.1 Function Documentation

2.17.1.1 [affect_value_st\(\)](#)

```
int affect_value_st (
    params_values_t * values,
    corewar_t * global,
    processes_t * process )
```

Validates and retrieves the values for the st instruction.

This function checks the argument types for the st instruction and retrieves their values. It ensures that the arguments are valid and updates the position in the arena accordingly.

Parameters

<i>values</i>	Pointer to the structure containing the parameter values.
<i>global</i>	Pointer to the <code>corewar</code> global structure.
<i>process</i>	Pointer to the current process structure.

Returns

0 on success, 1 if the arguments are invalid.

2.17.1.2 make_st()

```
int make_st (
    corewar_t * global,
    processes_t * process )
```

Executes the st instruction for a process.

This function decodes the st instruction, validates the argument types, retrieves their values, and stores the value from a register into another register or memory location. It also manages the readiness of the process and updates the program counter.

Parameters

<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.

Returns

0 on success, 1 if the operation is invalid.

2.17.1.3 stock_value()

```
void stock_value (
    corewar_t * global,
    processes_t * process,
    params_values_t values )
```

Stores a value in the arena or a register.

This function stores the value from a register into either another register or a memory location in the arena, depending on the argument type. The position in the arena is calculated using the offset.

Parameters

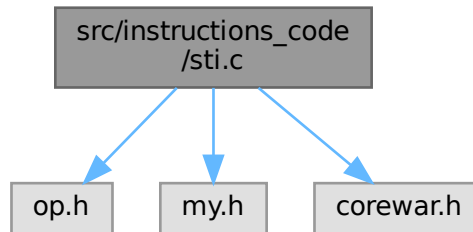
<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.
<i>values</i>	The structure containing the parameter values.

2.18 src/instructions_code/sti.c File Reference

```
#include "op.h"
#include "my.h"
```

```
#include "corewar.h"
```

Include dependency graph for sti.c:



Functions

- int [recup_value_sti](#) (params_values_t *values, corewar_t *global, processes_t *process)
Validates and retrieves the values for the sti instruction.
- void [stock_value_i](#) (params_values_t values, corewar_t *global, processes_t *process)
Stores a value in the arena for the sti instruction.
- int [make_sti](#) (corewar_t *global, processes_t *process)
Executes the sti instruction for a process.

2.18.1 Function Documentation

2.18.1.1 make_sti()

```
int make_sti (
    corewar_t * global,
    processes_t * process )
```

Executes the sti instruction for a process.

This function decodes the sti instruction, validates the argument types, retrieves their values, and stores the value from a register into a memory location in the arena. It also manages the readiness of the process and updates the program counter.

Parameters

<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.

Returns

0 on success, 1 if the operation is invalid.

2.18.1.2 recup_value_sti()

```
int recup_value_sti (
    params_values_t * values,
    corewar_t * global,
    processes_t * process )
```

Validates and retrieves the values for the sti instruction.

This function checks the argument types for the sti instruction and retrieves their values. It ensures that the arguments are valid and updates the position in the arena accordingly.

Parameters

<i>values</i>	Pointer to the structure containing the parameter values.
<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.

Returns

0 on success, 1 if the arguments are invalid.

2.18.1.3 stock_value_i()

```
void stock_value_i (
    params_values_t values,
    corewar_t * global,
    processes_t * process )
```

Stores a value in the arena for the sti instruction.

This function calculates the position in the arena based on the sum of the second and third arguments and stores the value from the specified register at that position.

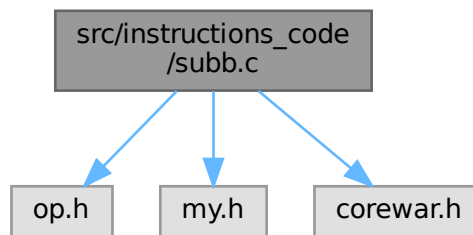
Parameters

<i>values</i>	The structure containing the parameter values.
<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.

2.19 src/instructions_code/subb.c File Reference

```
#include "op.h"
#include "my.h"
#include "corewar.h"
```

Include dependency graph for subb.c:



Functions

- int [affect_value_subb](#) (processes_t *process, params_values_t *values, corewar_t *global)
Executes the SUB operation for a process.
- int [make_sub](#) (corewar_t *global, processes_t *process)
Handles the SUB instruction for a process.

2.19.1 Function Documentation

2.19.1.1 [affect_value_subb\(\)](#)

```
int affect_value_subb (
    processes_t * process,
    params_values_t * values,
    corewar_t * global )
```

Executes the SUB operation for a process.

This function performs the subtraction of two register values and stores the result in a third register. It also updates the program counter and the carry flag of the process. The function ensures that the operation is valid by checking the argument types and waiting for the required number of cycles before execution.

Parameters

<i>process</i>	Pointer to the current process structure.
<i>values</i>	Pointer to the structure containing the parameter values.
<i>global</i>	Pointer to the corewar global structure.

Returns

0 on success, 1 if the operation is invalid.

2.19.1.2 make_sub()

```
int make_sub (
    corewar_t * global,
    processes_t * process )
```

Handles the SUB instruction for a process.

This function decodes the SUB instruction from the arena, validates the argument types, and calls the function to execute the operation. The SUB instruction subtracts the values of two registers and stores the result in a third register.

Parameters

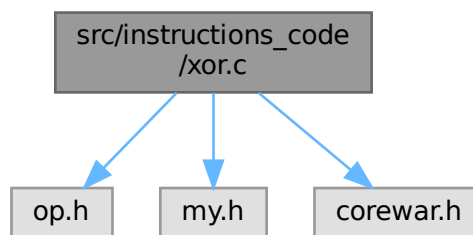
<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.

Returns

0 on success, 1 if the operation is invalid.

2.20 src/instructions_code/xor.c File Reference

```
#include "op.h"
#include "my.h"
#include "corewar.h"
Include dependency graph for xor.c:
```



Functions

- int [affect_value_xor](#) (processes_t *process, params_values_t *values, corewar_t *global)
Executes the XOR operation for a process.
- int [make_xor](#) (corewar_t *global, processes_t *process)
Handles the XOR instruction for a process.

2.20.1 Function Documentation

2.20.1.1 `affect_value_xor()`

```
int affect_value_xor (
    processes_t * process,
    params_values_t * values,
    corewar_t * global )
```

Executes the XOR operation for a process.

This function performs a bitwise XOR operation between two values and stores the result in a specified register. It also updates the program counter and the carry flag of the process. The function ensures that the operation is valid by checking the argument types and waiting for the required number of cycles before execution.

Parameters

<i>process</i>	Pointer to the current process structure.
<i>values</i>	Pointer to the structure containing the parameter values.
<i>global</i>	Pointer to the corewar global structure.

Returns

0 on success, 1 if the operation is invalid.

2.20.1.2 `make_xor()`

```
int make_xor (
    corewar_t * global,
    processes_t * process )
```

Handles the XOR instruction for a process.

This function decodes the XOR instruction from the arena, validates the argument types, and calls the function to execute the operation. The XOR instruction performs a bitwise XOR between two values and stores the result in a register.

Parameters

<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.

Returns

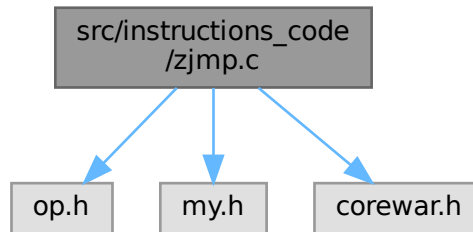
0 on success, 1 if the operation is invalid.

2.21 `src/instructions_code/zjmp.c` File Reference

```
#include "op.h"
#include "my.h"
```

```
#include "corewar.h"
```

Include dependency graph for zjmp.c:



Functions

- int [take_one_arg](#) (params_values_t *values, corewar_t *global)
Retrieves a single argument for the zjmp instruction.
- int [make_zjmp](#) (corewar_t *global, processes_t *process)
Executes the zjmp instruction for a process.

2.21.1 Function Documentation

2.21.1.1 make_zjmp()

```
int make_zjmp (
    corewar_t * global,
    processes_t * process )
```

Executes the zjmp instruction for a process.

This function decodes the zjmp instruction and checks the carry flag of the process. If the carry flag is set to 1, the program counter is updated to the specified offset. Otherwise, the program counter is incremented to the next instruction. The function also manages the readiness of the process.

Parameters

<i>global</i>	Pointer to the corewar global structure.
<i>process</i>	Pointer to the current process structure.

Returns

0 on success.

2.21.1.2 take_one_arg()

```
int take_one_arg (
```

```
params_values_t * values,
corewar_t * global )
```

Retrieves a single argument for the zjmp instruction.

This function retrieves the direct value of the argument for the zjmp instruction. The value is restricted by IDX_MOD to ensure it is within the valid range.

Parameters

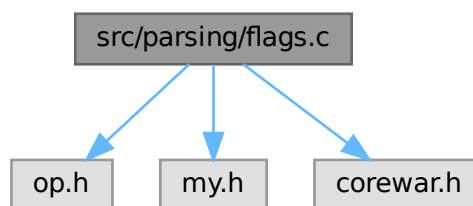
<i>values</i>	Pointer to the structure containing the parameter values.
<i>global</i>	Pointer to the corewar global structure.

Returns

The direct value of the argument, restricted by IDX_MOD.

2.22 src/parsing/flags.c File Reference

```
#include "op.h"
#include "my.h"
#include "corewar.h"
Include dependency graph for flags.c:
```



Functions

- int [change_prog_number](#) (char **av, player_t player[4], int i, int actual_champion)
Changes the program number for a champion.
- int [change_load_address](#) (char **av, player_t player[4], int i, int actual_champion)
Changes the load address for a champion.
- int [verif_cor](#) (char **av, int i)
Verifies if a file has the .cor extension.
- static void [assign_missing_addresses](#) (player_t player[4], int count)
- static void [assign_missing_numbers](#) (player_t player[4], int count)
- int [recup_load_and_number](#) (char **av, player_t player[4], int ac, parsing_position_t *pars)
Parses and assigns load addresses and program numbers.
- static int [change_number_cycle](#) (char **av, parsing_position_t *pars, corewar_t *global)
- static int [recover_cycle](#) (char **av, int ac, parsing_position_t *pars, corewar_t *global)
- static int [verif_end_parsing](#) (parsing_position_t pars, corewar_t *global, player_t player[4])
- int [parsing_flags](#) (int ac, char **av, player_t player[4], corewar_t *global)
Parses the flags and initializes the players and global data.

2.22.1 Function Documentation

2.22.1.1 assign_missing_addresses()

```
static void assign_missing_addresses (
    player_t player[4],
    int count ) [static]
```

2.22.1.2 assign_missing_numbers()

```
static void assign_missing_numbers (
    player_t player[4],
    int count ) [static]
```

2.22.1.3 change_load_adress()

```
int change_load_adress (
    char ** av,
    player_t player[4],
    int i,
    int actual_champion )
```

Changes the load address for a champion.

This function validates and assigns a load address to a champion based on the provided argument. It ensures that the argument is a valid number.

Parameters

<i>av</i>	Array of arguments.
<i>player</i>	Array of player structures.
<i>i</i>	Index of the current argument.
<i>actual_champion</i>	Index of the current champion.

Returns

0 on success, 84 if the argument is invalid.

2.22.1.4 change_number_cycle()

```
static int change_number_cycle (
    char ** av,
    parsing_position_t * pars,
    corewar_t * global ) [static]
```

2.22.1.5 `change_prog_number()`

```
int change_prog_number (
    char ** av,
    player_t player[4],
    int i,
    int actual_champion )
```

Changes the program number for a champion.

This function validates and assigns a program number to a champion based on the provided argument. It ensures that the argument is a valid number.

Parameters

<i>av</i>	Array of arguments.
<i>player</i>	Array of player structures.
<i>i</i>	Index of the current argument.
<i>actual_champion</i>	Index of the current champion.

Returns

0 on success, 84 if the argument is invalid.

2.22.1.6 `parsing_flags()`

```
int parsing_flags (
    int ac,
    char ** av,
    player_t player[4],
    corewar_t * global )
```

Parses the flags and initializes the players and global data.

This function processes the command-line arguments to parse the flags, validate the input, and initialize the player and global structures.

Parameters

<i>ac</i>	Number of arguments.
<i>av</i>	Array of arguments.
<i>player</i>	Array of player structures.
<i>global</i>	Pointer to the corewar global structure.

Returns

0 on success, 84 if an error occurs.

2.22.1.7 `recover_cycle()`

```
static int recover_cycle (
```

```
char ** av,  
int ac,  
parsing_position_t * pars,  
corewar_t * global ) [static]
```

2.22.1.8 recup_load_and_number()

```
int recup_load_and_number (  
    char ** av,  
    player_t player[4],  
    int ac,  
    parsing_position_t * pars )
```

Parses and assigns load addresses and program numbers.

This function processes the arguments to assign load addresses and program numbers to the champions. It also verifies the validity of the provided arguments.

Parameters

<i>av</i>	Array of arguments.
<i>player</i>	Array of player structures.
<i>ac</i>	Number of arguments.
<i>pars</i>	Pointer to the parsing position structure.

Returns

0 on success, 84 if an error occurs.

2.22.1.9 verif_cor()

```
int verif_cor (  
    char ** av,  
    int i )
```

Verifies if a file has the .cor extension.

This function checks if the provided file name has the correct .cor extension, indicating that it is a valid Corewar binary file.

Parameters

<i>av</i>	Array of arguments.
<i>i</i>	Index of the current argument.

Returns

0 if the file is valid, 1 otherwise.

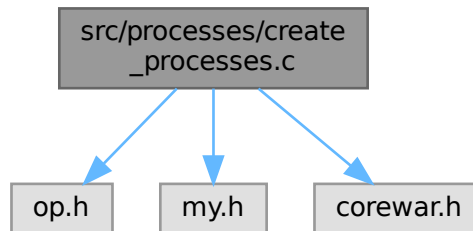
2.22.1.10 `verif_end_parsing()`

```
static int verif_end_parsing (
    parsing_position_t pars,
    corewar_t * global,
    player_t player[4] ) [static]
```

2.23 `src/processes/create_processes.c` File Reference

```
#include "op.h"
#include "my.h"
#include "corewar.h"
```

Include dependency graph for `create_processes.c`:



Functions

- void [affect_value_process](#) (processes_t *node, int i, corewar_t global)
Initializes the values of a process node.
- int [initialize_process](#) (processes_t **instr, corewar_t global, int i)
Creates and initializes a new process.

2.23.1 Function Documentation

2.23.1.1 `affect_value_process()`

```
void affect_value_process (
    processes_t * node,
    int i,
    corewar_t global )
```

Initializes the values of a process node.

This function assigns initial values to a process node, including the player ID, program counter, carry flag, registers, and other attributes.

Parameters

<i>node</i>	Pointer to the process node to initialize.
<i>i</i>	The index of the player associated with the process.
<i>global</i>	The corewar global structure containing player data.

2.23.1.2 initialize_process()

```
int initialize_process (
    processes_t ** instr,
    corewar_t global,
    int i )
```

Creates and initializes a new process.

This function allocates memory for a new process, initializes its values, and adds it to the linked list of processes. If the list is empty, the new process becomes the head of the list.

Parameters

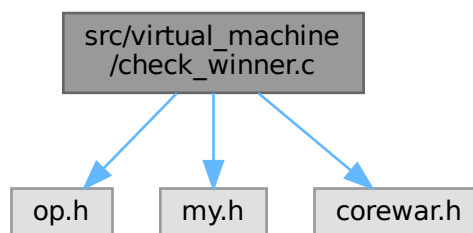
<i>instr</i>	Pointer to the head of the process list.
<i>global</i>	The corewar global structure containing player data.
<i>i</i>	The index of the player associated with the process.

Returns

0 on success, 84 if memory allocation fails.

2.24 src/virtual_machine/check_winner.c File Reference

```
#include "op.h"
#include "my.h"
#include "corewar.h"
Include dependency graph for check_winner.c:
```



Functions

- void `check_delta` (`corewar_t *global`, `processes_t *proc_current`, `processes_t *prev`, `processes_t *next`)
Checks if a process should be removed due to inactivity.
- void `print_winner` (`corewar_t *global`)
Prints the winner of the Corewar game.
- void `check_hunger_games` (`corewar_t *global`)
Manages the elimination of inactive processes.

2.24.1 Function Documentation

2.24.1.1 `check_delta()`

```
void check_delta (
    corewar_t * global,
    processes_t * proc_current,
    processes_t * prev,
    processes_t * next )
```

Checks if a process should be removed due to inactivity.

This function verifies if a process has exceeded the allowed number of cycles without a live signal. If so, it removes the process from the process list.

Parameters

<i>global</i>	Pointer to the corewar global structure.
<i>proc_current</i>	Pointer to the current process being checked.
<i>prev</i>	Pointer to the previous process in the list.
<i>next</i>	Pointer to the next process in the list.

2.24.1.2 `check_hunger_games()`

```
void check_hunger_games (
    corewar_t * global )
```

Manages the elimination of inactive processes.

This function iterates through the process list and removes processes that have not sent a live signal within the allowed number of cycles. It also adjusts the cycle-to-die value if the number of live signals exceeds the threshold.

Parameters

<i>global</i>	Pointer to the corewar global structure.
---------------	--

2.24.1.3 `print_winner()`

```
void print_winner (
    corewar_t * global )
```

Prints the winner of the Corewar game.

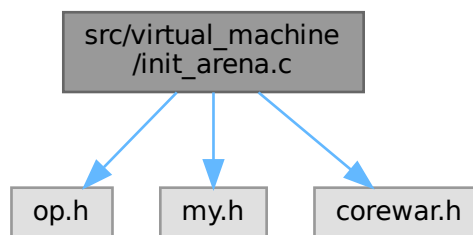
This function displays the winner of the game based on the last process to send a live signal. If no valid winner exists, it prints an empty message.

Parameters

<i>global</i>	Pointer to the corewar global structure.
---------------	--

2.25 src/virtual_machine/init_arena.c File Reference

```
#include "op.h"
#include "my.h"
#include "corewar.h"
Include dependency graph for init_arena.c:
```



Functions

- int [verif_size_arena](#) (int i, corewar_t *global)
Verifies and places a player's code in the arena.
- int [create_arena_with_player](#) (corewar_t *global)
Initializes the arena and places players' codes.

2.25.1 Function Documentation

2.25.1.1 create_arena_with_player()

```
int create_arena_with_player (
    corewar_t * global )
```

Initializes the arena and places players' codes.

This function allocates memory for the arena and initializes it to 0. It then iterates through all players and places their codes into the arena at their respective load addresses. If any placement fails, the function returns an error.

Parameters

<i>global</i>	Pointer to the corewar global structure.
---------------	--

Returns

0 on success, 84 if an error occurs.

2.25.1.2 `verif_size_arena()`

```
int verific_size_arena (
    int i,
    corewar_t * global )
```

Verifies and places a player's code in the arena.

This function checks if the player's code can fit into the arena at the specified load address. If the placement is valid, the player's code is copied into the arena. Otherwise, an error is returned.

Parameters

<i>i</i>	The index of the player to verify.
<i>global</i>	Pointer to the corewar global structure.

Returns

0 on success, 84 if the placement is invalid.

Index

- add.c
 - affect_value_add, [9](#)
 - make_add, [10](#)
- add_new_process
 - fork.c, [16](#)
- aff.c
 - make_aff, [11](#)
 - recover_registre, [11](#)
- affect_value_add
 - add.c, [9](#)
- affect_value_and
 - and.c, [12](#)
- affect_value_ld
 - ld.c, [18](#)
- affect_value_ldi
 - ldi.c, [20](#)
- affect_value_lld
 - lld.c, [24](#)
- affect_value_lldi
 - lldi.c, [25](#)
- affect_value_or
 - or.c, [29](#)
- affect_value_process
 - create_processes.c, [42](#)
- affect_value_st
 - st.c, [30](#)
- affect_value_subb
 - subb.c, [34](#)
- affect_value_xor
 - xor.c, [36](#)
- and.c
 - affect_value_and, [12](#)
 - make_and, [13](#)
- assign_missing_addresses
 - flags.c, [39](#)
- assign_missing_numbers
 - flags.c, [39](#)
- calculate_new_pc
 - modif_proces.c, [27](#)
- change_carry
 - modif_proces.c, [27](#)
- change_load_adress
 - flags.c, [39](#)
- change_number_cycle
 - flags.c, [39](#)
- change_prog_number
 - flags.c, [39](#)
- check_delta
 - check_winner.c, [44](#)
- check_hunger_games
 - check_winner.c, [44](#)
- check_if_indirect
 - check_types.c, [14](#)
- check_type
 - check_types.c, [14](#)
- check_types.c
 - check_if_indirect, [14](#)
 - check_type, [14](#)
 - get_arg_type, [15](#)
 - recover_indirect, [15](#)
- check_winner.c
 - check_delta, [44](#)
 - check_hunger_games, [44](#)
 - print_winner, [44](#)
- create_arena_with_player
 - init_arena.c, [45](#)
- create_champions
 - create_champions.c, [6](#)
- create_champions.c
 - create_champions, [6](#)
 - get_champion_infos, [7](#)
 - load_champion_body_to_buffer, [7](#)
 - reverse_bytes, [7](#)
- create_processes.c
 - affect_value_process, [42](#)
 - initialize_process, [43](#)
- execute_cycle
 - main.c, [4](#)
- flags.c
 - assign_missing_addresses, [39](#)
 - assign_missing_numbers, [39](#)
 - change_load_adress, [39](#)
 - change_number_cycle, [39](#)
 - change_prog_number, [39](#)
 - parsing_flags, [40](#)
 - recover_cycle, [40](#)
 - recup_load_and_number, [41](#)
 - verif_cor, [41](#)
 - verif_end_parsing, [41](#)
- fork.c
 - add_new_process, [16](#)
 - make_fork, [17](#)
- free.c
 - free_struct, [8](#)
- free_struct
 - free.c, [8](#)

- get_arg_type
 - check_types.c, 15
- get_champion_infos
 - create_champions.c, 7
- init_arena.c
 - create_arena_with_player, 45
 - verif_size_arena, 46
- initialize_process
 - create_processes.c, 43
- launch_vm
 - main.c, 4
- ld.c
 - affect_value_ld, 18
 - make_ld, 18
 - print_register_change, 18
- ldi.c
 - affect_value_ldi, 20
 - make_ldi, 20
 - read_value, 20
- lfork.c
 - make_lfork, 22
- live.c
 - make_live, 23
- lld.c
 - affect_value_lld, 24
 - make_lld, 24
- lldi.c
 - affect_value_lddi, 25
 - make_lddi, 25
- load_champion_body_to_buffer
 - create_champions.c, 7
- main
 - main.c, 4
- main.c, 3
 - execute_cycle, 4
 - launch_vm, 4
 - main, 4
 - make_instruction, 4
 - print_help, 5
 - step_forward, 5
- make_add
 - add.c, 10
- make_aff
 - aff.c, 11
- make_and
 - and.c, 13
- make_fork
 - fork.c, 17
- make_instruction
 - main.c, 4
- make_ld
 - ld.c, 18
- make_ldi
 - ldi.c, 20
- make_lfork
 - lfork.c, 22
- make_live
 - live.c, 23
- make_lld
 - lld.c, 24
- make_lddi
 - lldi.c, 25
- make_or
 - or.c, 29
- make_st
 - st.c, 31
- make_sti
 - sti.c, 32
- make_sub
 - subb.c, 34
- make_xor
 - xor.c, 36
- make_zjmp
 - zjmp.c, 37
- modif_proces.c
 - calculate_new_pc, 27
 - change_carry, 27
 - recover_dir_size, 27
 - recover_direct_value, 28
- or.c
 - affect_value_or, 29
 - make_or, 29
- parsing_flags
 - flags.c, 40
- print_help
 - main.c, 5
- print_register_change
 - ld.c, 18
- print_winner
 - check_winner.c, 44
- read_value
 - ldi.c, 20
- recover_cycle
 - flags.c, 40
- recover_dir_size
 - modif_proces.c, 27
- recover_direct_value
 - modif_proces.c, 28
- recover_indirect
 - check_types.c, 15
- recover_registre
 - aff.c, 11
- recup_load_and_number
 - flags.c, 41
- recup_value_sti
 - sti.c, 32
- reverse_bytes
 - create_champions.c, 7
- src/champions/create_champions.c, 6
- src/free_functions/free.c, 8
- src/instructions_code/add.c, 9

- src/instructions_code/aff.c, [10](#)
- src/instructions_code/and.c, [12](#)
- src/instructions_code/check_types.c, [13](#)
- src/instructions_code/fork.c, [16](#)
- src/instructions_code/ld.c, [17](#)
- src/instructions_code/ldi.c, [19](#)
- src/instructions_code/lfork.c, [21](#)
- src/instructions_code/live.c, [22](#)
- src/instructions_code/lld.c, [23](#)
- src/instructions_code/lldi.c, [24](#)
- src/instructions_code/modif_proces.c, [26](#)
- src/instructions_code/or.c, [28](#)
- src/instructions_code/st.c, [30](#)
- src/instructions_code/sti.c, [31](#)
- src/instructions_code/subb.c, [33](#)
- src/instructions_code/xor.c, [35](#)
- src/instructions_code/zjmp.c, [36](#)
- src/parsing/flags.c, [38](#)
- src/processes/create_processes.c, [42](#)
- src/virtual_machine/check_winner.c, [43](#)
- src/virtual_machine/init_arena.c, [45](#)
- st.c
 - affect_value_st, [30](#)
 - make_st, [31](#)
 - stock_value, [31](#)
- step_forward
 - main.c, [5](#)
- sti.c
 - make_sti, [32](#)
 - recup_value_sti, [32](#)
 - stock_value_i, [33](#)
- stock_value
 - st.c, [31](#)
- stock_value_i
 - sti.c, [33](#)
- subb.c
 - affect_value_subb, [34](#)
 - make_sub, [34](#)
- take_one_arg
 - zjmp.c, [37](#)
- verif_cor
 - flags.c, [41](#)
- verif_end_parsing
 - flags.c, [41](#)
- verif_size_arena
 - init_arena.c, [46](#)
- xor.c
 - affect_value_xor, [36](#)
 - make_xor, [36](#)
- zjmp.c
 - make_zjmp, [37](#)
 - take_one_arg, [37](#)